

Threat Intelligence Platform Architecture Design

1. Introduction

In a modern enterprise Security Operations Center (SOC), a robust Threat Intelligence (TI) enrichment architecture is critical for rapid detection and response. This document outlines the design of a scalable TI platform that aggregates, normalizes, enriches, and operationalizes Indicators of Compromise (IOCs). The architecture integrates multiple internal and external data sources, orchestrates enrichment through a centralized engine, and seamlessly feeds actionable intelligence into SOC workflows (e.g., SIEM, SOAR, and EDR systems).

The reference framework for this design is loosely based on the capabilities provided by the MISP (Malware Information Sharing Platform) Project, customized to handle enterprise-scale telemetry and automated response requirements.

2. Architecture Components

2.1. Data Sources

The foundation of the TI platform is its intelligence feeds, divided into internal and external sources:

- * **Internal Data Sources:** Network telemetry, firewall logs, IDS/IPS alerts, endpoint detection data (EDR), and internal incident response reports. These provide the context for what is actively targeting the enterprise.
- * **External Data Sources:** Commercial feeds (e.g., Recorded Future, CrowdStrike), open-source intelligence (OSINT) such as AlienVault OTX, and specialized enrichment APIs (VirusTotal, AbuseIPDB, URLhaus). These provide global context, reputation scores, and attribution data.

2.2. Enrichment Engine

The Enrichment Engine acts as the central brain of the platform. When a raw IOC (e.g., an IP address, domain, or file hash) is ingested, the engine queries various internal and external sources to gather context. It performs normalization to standard formats (such as STIX/TAXII)

and correlates data across multiple feeds to build a comprehensive risk profile for the indicator.

2.3. IOC Database (Threat Intelligence Platform - TIP)

The enriched IOCs, along with their metadata (first seen, last seen, confidence score, source diversity), are stored in the IOC Database. This database acts as the single source of truth for the enterprise. It supports high-performance querying and is designed to handle rapid read/write operations essential for real-time blocking and retrospective hunting.

2.4. Workflow Integration

The final component involves operationalizing the intelligence. The platform integrates via APIs with:

- * **SIEM (Security Information and Event Management)**: Pushing high-confidence IOCs for real-time alerting and historical log correlation.
- * **SOAR (Security Orchestration, Automation, and Response)**: Triggering automated playbooks (e.g., automatically isolating a host or blocking a malicious IP at the perimeter).
- * **Firewalls/EDR**: Generating dynamic blocklists.

3. Architecture Diagram and Data Flow

```
graph TD
    subgraph External_Sources [External Sources]
        VT[VirusTotal API]
        AB[AbuseIPDB API]
        OTX[AlienVault OTX]
        OSINT[OSINT Feeds]
    end

    subgraph Internal_Sources [Internal Sources]
        SIEM_Logs[SIEM / Logs]
        EDR_Alerts[EDR Telemetry]
        IR[Incident Response]
    end

    subgraph TI_Platform_Core [TI Platform Core]
        Ingest[Ingestion & Normalization Layer]
        Engine[Enrichment Engine]
        Cache[(Local Query Cache)]
        DB[(IOC Database)]
        Lifecycle[Lifecycle Manager]
    end
```

```

subgraph Workflow Integration
    SIEM_Export[SIEM Blocklist]
    SOAR_Playbook[SOAR Automation]
    Analyst[SOC Analyst Dashboard]
end

Internal Sources --> Ingest
OSINT --> Ingest
Ingest --> Engine
Engine <--> Cache
Engine --> VT
Engine --> AB
Engine --> OTX

VT --> Engine
AB --> Engine
OTX --> Engine

Engine --> DB
Lifecycle <--> DB

DB --> SIEM_Export
DB --> SOAR_Playbook
DB --> Analyst

```

Data Flow Overview:

1. **Ingestion:** Raw IOCs are collected from internal sensors and external OSINT feeds.
2. **Enrichment:** The Enrichment Engine receives the IOC and first checks the Local Query Cache. If a cache miss occurs (or the cache is stale), it queries external APIs like VirusTotal and AbuseIPDB.
3. **Storage:** The enriched data, including risk scores and context, is saved to the IOC Database.
4. **Operationalization:** The database pushes automated blocklists to the SIEM and triggers playbooks in the SOAR platform.
5. **Maintenance:** The Lifecycle Manager periodically scans the database to decay scores or expire old IOCs.

4. Key Engineering Challenges and Solutions

4.1. Rate Limit Handling

Querying multiple external APIs, especially on free or tiered tiers, introduces the risk of rate limiting (HTTP 429 Too Many Requests). * **Caching:** Implementing a local cache (e.g., Redis or in-memory JSON) ensures that an indicator queried recently is not re-queried against the external API until a predefined Time-To-Live (TTL) expires. * **Batching and Throttling:** The Enrichment Engine utilizes asynchronous queues with token bucket or leaky bucket algorithms to throttle outgoing requests, ensuring they remain within the provider's limits. * **Exponential Backoff:** If a rate limit is hit, the engine implements exponential backoff with jitter before retrying the request.

4.2. IOC Expiration and Lifecycle Management

Threat intelligence decays rapidly; an IP address hosting malware today might be reassigned to a legitimate service tomorrow. Retaining stale IOCs leads to bloated databases and false positives. * **Automated Expiry:** Every IOC is assigned a `last_seen` timestamp and an expiration date based on its type. For example, IPv4 addresses might expire in 7-14 days, while file hashes (SHA256) might be retained permanently. * **Decay Models:** Instead of immediate deletion, confidence scores decay over time. If an IOC is not sighted within a specific window, its confidence drops below the threshold required for automated blocking, downgrading it to a "monitoring only" state.

4.3. False Positive Prevention

Automatically blocking infrastructure based on threat feeds can inadvertently block legitimate business traffic (e.g., blocking an AWS NAT Gateway or Google DNS). * **Safelist / Allowlist Architecture:** The platform implements a robust, manually curated global allowlist. Before any IOC is marked for blocking, it is checked against this list (containing RFC 1918 addresses, major CDN IP ranges, and known good domains). * **Multi-Source Validation:** An IOC requires validation from multiple independent sources to reach a high confidence score. A rule-based system aggregates the scores (e.g., an IP must be flagged by both VirusTotal and AbuseIPDB to exceed the blocking threshold). * **Score Stability:** Sudden spikes in risk scores for previously trusted indicators trigger manual analyst review rather than automated blocking.

5. Conclusion

This TI platform architecture provides a resilient, scalable, and automated approach to threat intelligence. By intelligently managing external API interactions through caching and rate limiting, applying strict lifecycle controls to prevent database bloat, and utilizing allowlisting

and multi-source correlation to minimize false positives, the SOC is empowered to respond to threats with high confidence and speed.